

L04-P03 练习手册

大模型微调原理和实践

实践 1：深度学习基础示例

本实践演示如何使用 PyTorch 构建并训练一个前馈神经网络，实现对 $\sin(x)$ 函数的拟合。通过这个简洁的例子，可以快速理解深度学习训练的完整流程。

核心流程

步骤	内容说明
1. 生成训练数据	在区间 $[-2\pi, 2\pi]$ 内均匀生成 2000 个样本点，计算对应的 $\sin(x)$ 作为目标值
2. 创建网络模型	搭建三层网络：1 $\rightarrow$ 8 $\rightarrow$ 8 $\rightarrow$ 1，每层使用 Tanh 激活函数
3. 训练模型	采用 MSE 损失函数 + Adam 优化器，batch size=200，训练 200 轮
4. 结果展示	绘制真实 $\sin(x)$ 与模型预测曲线对比图
5. 性能分析	绘制训练损失曲线，观察模型学习收敛过程

网络结构

输入层(1) $\rightarrow$ 隐藏层 1(8) $\rightarrow$ 隐藏层 2(8) $\rightarrow$ 输出层(1)

- 激活函数：Tanh
- 优化器：Adam (lr=0.01)
- 损失函数：MSE

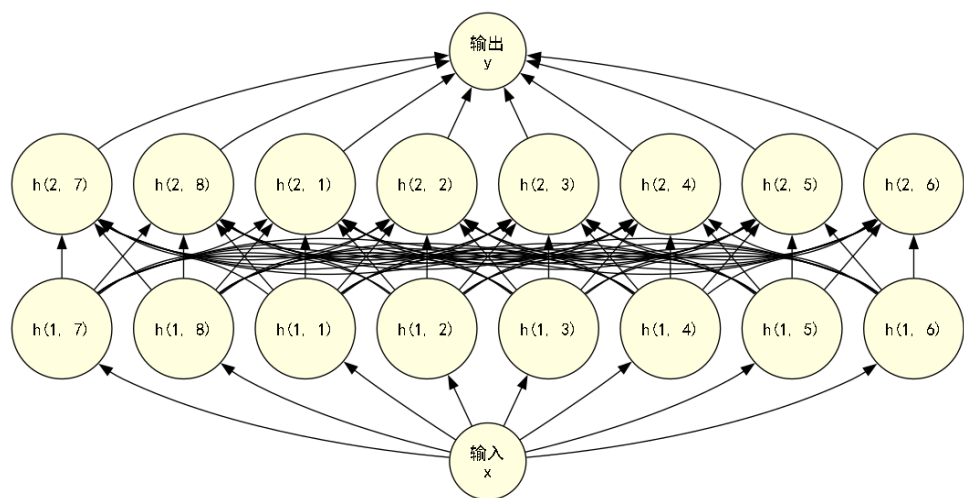


图 1. 网络结构示意图

实践 2：LoRA 原理演示

LoRA (Low-Rank Adaptation) 是一种参数高效微调方法，通过在冻结的预训练权重旁添加低秩可训练矩阵，大幅减少训练参数量，同时保持接近全参微调的效果。

2.1 LoRA 实现示例

程序：lora_sample.py 运行：python lora_sample.py

(1) 模块初始化

- 接收输入维度、输出维度、秩 r 、缩放因子和 dropout 参数
- 定义并冻结原始权重 W ，不参与训练
- 新增两个可训练的小矩阵 A 和 B ，构建低秩近似
- 设置缩放因子，控制更新幅度；配置可选 dropout

(2) 前向传播

- 用冻结的 W 计算基线输出
- 输入经过矩阵 A 得到低秩表示，再经矩阵 B 恢复到输出维度
- 对更新部分进行缩放，与基线输出相加得到最终结果

(3) 训练过程

- 只有 A 、 B 矩阵是可训练参数， W 保持冻结状态
- 大幅减少可训练参数量，节省显存和计算资源

(4) 权重合并与反合并

- 推理时可将 LoRA 更新合并进 W ，前向传播只走 W 路径，速度更快
- 如需继续训练，可将更新反向移除，恢复到原始状态

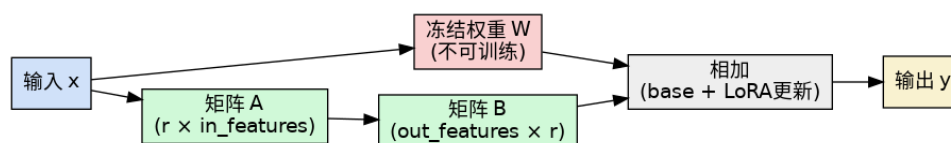


图 2. LoRA 原理示意图

2.2 基座模型与微调模型参数对比

`程序:` show_model_size.py `运行:` python show_model_size.py

该程序加载 Qwen1.5-1.8B-Chat 模型，分别打印 LoRA 注入前后的参数规模对比，直观展示 LoRA 只让很少的参数可训练，其余保持冻结。

指标	基座模型（LoRA 之前）	LoRA 模型（应用后）
总参数量	1,836.83 M	1,838.40 M
可训练参数量	1,836.83 M（100%）	1.57 M（≈0.09%）

可见，应用 LoRA 后可训练参数仅占总参数的约 0.09%，显著降低了计算和存储开销。

实践 3：Swift 命令行训练

Swift 是阿里云开源的大模型微调框架，支持 LoRA、QLoRA、全参微调等多种训练方式。本实践使用命令行方式对 Qwen2.5-7B-Instruct 进行 LoRA 微调。

环境安装

```
pip install ms-swift==3.12
pip install -U "gradio==4.44.1" "gradio_client==1.3.0" "pydantic==2.10.6"
```

启动训练

```
CUDA_VISIBLE_DEVICES=0 swift sft \
  --model Qwen/Qwen2.5-7B-Instruct \
  --train_type lora \
  --dataset 'AI-ModelScope/alpaca-gpt4-data-zh#500' \
    'AI-ModelScope/alpaca-gpt4-data-en#500' \
    'swift/self-cognition#500' \
  --torch_dtype bfloat16 --num_train_epochs 1 \
  --per_device_train_batch_size 1 --learning_rate 1e-4 \
  --lora_rank 8 --lora_alpha 32 --target_modules all-linear \
  --gradient_accumulation_steps 16 --output_dir output \
  --system 'You are a helpful assistant.'
```

参数详解

1. 模型与微调方式

参数	值	说明
CUDA_VISIBLE_DEVICES	0	指定使用第 0 号 GPU，多卡可改为 0,1
swift sft	—	调用 Swift 的监督式微调（SFT）子命令
--model	Qwen/Qwen2.5-7B-Instruct	基座模型，支持 HuggingFace/ModelScope 地址或

参数	值	说明
		本地路径
--train_type	lora	训练方式：只训练附加的低秩矩阵，冻结原始权重

2. 数据集设置

数据集	样本量	用途
AI-ModelScope/alpaca-gpt4-data-zh	500 条	中文 GPT-4 指令数据
AI-ModelScope/alpaca-gpt4-data-en	500 条	英文 GPT-4 指令数据
swift/self-cognition	500 条	自我认知/工具使用/聊天人格

总样本量约 1500 条，适合演示与快速试验。中英文数据混合可让模型同时练习双语问答能力。

3. 训练超参数

参数	值	说明
--torch_dtype	bfloat16	BF16 精度训练，比 FP16 更稳定，显存占用比 FP32 小
--num_train_epochs	1	训练 1 个 epoch，适合快速演示
--per_device_train_batch_size	1	每张 GPU 的训练 batch size，配合梯度累积模拟大 batch
--learning_rate	1e-4	学习率，LoRA 微调的常用值
--gradient_accumulation_steps	16	梯度累积 16 步后更新，等价有效 batch size = 16
--max_length	2048	单条样本最大 token 长度，超出截断

4. LoRA 相关参数

参数	值	说明
--lora_rank	8	秩 $r=8$, 7B 模型的常用配置, 平衡能力与参数量
--lora_alpha	32	缩放因子 $\alpha=32$, 控制 LoRA 更新幅度
--target_modules	all-linear	作用于所有线性层, 比只改 q_proj/v_proj 更强

实战建议： 医疗等高风险领域建议改为只在 q_proj、v_proj、o_proj 加 LoRA, 更加稳健。

5. 其他训练控制参数

参数	值	说明
--eval_steps	50	每 50 步做一次验证
--save_steps / --save_total_limit	50 / 2	每 50 步保存 checkpoint, 最多保留 2 个
--logging_steps	5	每 5 步打印日志, 便于观察 loss 变化
--warmup_ratio	0.05	前 5% 训练步数学习率从 0 线性增长, 避免剧烈更新
--dataloader_num_workers	4	DataLoader 使用 4 个子进程预取数据
--system	You are a helpful assistant.	训练时的 System Prompt, 影响模型角色风格

实践 4：Swift Web UI 可视化训练

Swift 提供了基于 Gradio 的 Web UI 界面，支持通过浏览器进行可视化训练配置与监控，降低了命令行的使用门槛。

环境与启动

```
pip install ms-swift==3.12
pip install gradio gradio_client pydantic
```

```
SWIFT_UI_LANG=zh swift web-ui
```

启动后默认地址为 <http://127.0.0.1:7860>，容器部署后可通过外部地址访问。

模型设置

模型 ID 或路径 (--model)

- 支持 ModelScope/HuggingFace 在线名称或本地路径
- 已下载到本地的模型优先使用本地路径，加载更快、更稳定

模型类型 (--model_type)

- Swift 内部的适配器类型，决定权重加载、LoRA adapter 构建、分词器选择等

示例： Qwen2.5 → qwen2_5, InternLM2 → internlm2, LLaMA → llama/llama2

模型 Prompt 模板类型 (--template)

- 训练/推理时 Swift 按模板拼接对话文本，选错会导致微调效果明显变差
- 必须与基座模型匹配：换模型时记得同步修改

训练记录与 System Prompt

- **训练记录：**可查看/复用之前的配置，删除记录仅删配置，不删权重文件
- **System 字段：**默认注入的 System Prompt，影响模型角色和对话风格

实战建议：企业内部知识助手可改为：“你是企业内部知识助手，请优先参考公司知识库回答。”

数据集设置

参数	示例值	说明
--dataset	AI-ModelScope/alpaca-gpt4-data-zh	支持远程或本地数据集，可多选拼接
--split_dataset_ratio	0.1	未标注 train/val 时自动按 10% 拆分验证集
--max_length	2048	每条样本最大 token 数，直接影响显存占用
--padding_free	开启	将 batch 内样本拼接训练，避免 padding 浪费，节省显存

实战建议： 7B 模型 + 24G 显卡 + LoRA 建议 max_length 从 2048 开始，有需要再拉长。

训练参数设置

参数	示例值	说明
--train_type	lora	lora/qlora 参数高效，full 全参显存极大
--seed	42	随机数种子，保证结果可复现
--torch_dtype	bfloat16	A100/H100/4090 建议 bfloat16，老显卡用 float16
--use_liger_kernel	关闭	融合算子优化，遇到 CUDA 报错时先关闭排查
DDP 分片数量	1	单卡训练设为 1；多卡设为对应 GPU 数
--deepspeed	—	LoRA+7B 基本不需要，大模型多卡训练时使用

Web UI 演示推荐参数

分类	参数	建议设置
模型设置	模型 ID	Qwen/Qwen2.5-7B-Instruct
	模型类型	自动识别或 qwen2_5
	Prompt 模板	默认或 qwen2_5
训练数据	数据集	AI-ModelScope/alpaca-gpt4-data-zh#200
	验证集拆分	0.1
	max_length	默认 (2048)
训练参数	训练方式	lora
	GPU	0 (单卡)
	Epoch	1
	Learning rate	5e-5
	LoRA rank / alpha	8 / 16
System 提示语	—	你是一个专业助手，请简洁回答。
输出目录	—	output_demo

实践 5：微调程序及训练过程监测

本实践展示如何通过 TensorBoard 和自定义日志绘图程序来监控 LoRA 微调过程中的损失变化，并根据不同情况制定相应的优化策略。

程序流程

步骤	内容
1	加载数据集
2	加载基座模型和分词器
3	配置 LoRA
4	数据预处理
5	设置训练参数
6	定义 Data Collator
7	自定义 Callback（显存监控）
8	定义 Trainer 并启动训练
9	保存 LoRA Adapter
10	推理测试

运行方式

```
python lora_train.py
```

监控工具

TensorBoard 实时监控

```
tensorboard --logdir=./logs --port=7006 --bind_all
```

启动后可通过浏览器访问 TensorBoard 界面，实时观察训练过程中的 loss、learning rate 等指标。

日志可视化绘图

```
python plot_log.py
```

使用自定义程序绘制 Training Loss 和 Evaluation Loss 曲线图，用于训练后的效果分析。

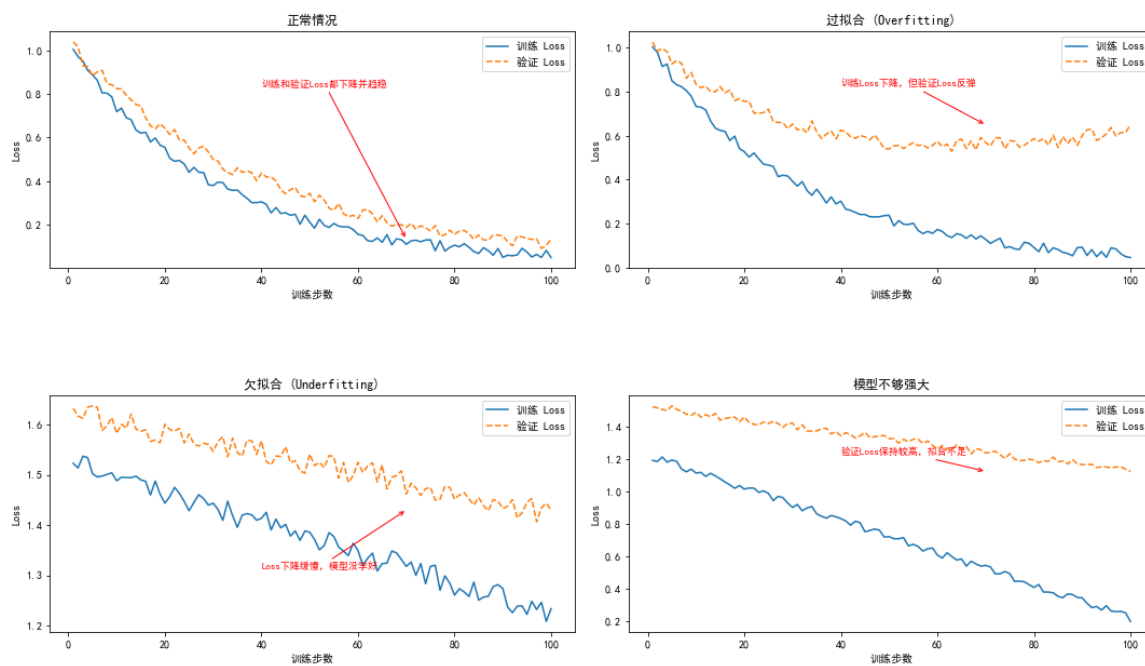


图 3. 训练损失曲线示例一

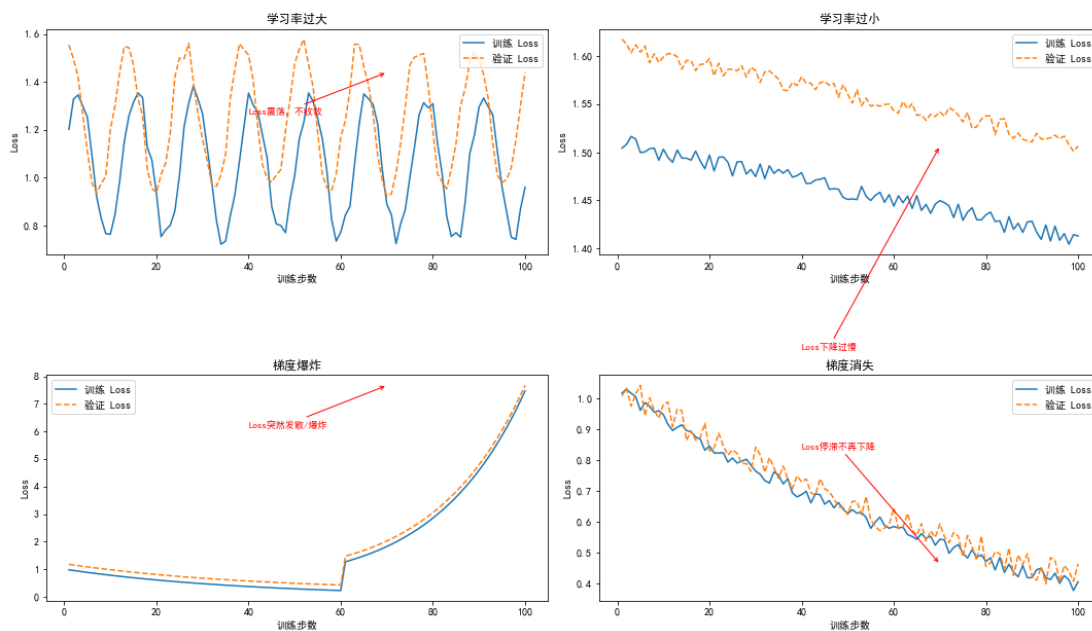


图 4. 训练损失曲线示例二

训练情况分析对策

通过观察 Training Loss 和 Evaluation Loss 的变化趋势，可以识别不同的训练问题并采取相应措施。

情况	现象	解决办法
正常	Train Loss 和 Eval Loss 均稳定下降	无需调整，训练过程健康
过拟合	Train Loss 持续下降，Eval Loss 反弹	增加正则化（dropout、weight decay）、数据增强、早停
欠拟合	Loss 下降不明显	增加模型容量、加 epoch、提高学习率、减少正则化
模型不够强大	Train Loss 有所下降，Eval Loss 始终较高	换更大模型、改进数据质量
学习率过大	Loss 剧烈震荡，不收敛	降低学习率、加调度器、梯度裁剪
学习率过小	Loss 下降极慢	提高学习率、加调度器、加 epoch
梯度爆炸	Loss 突然变得非常大或 NaN	梯度裁剪（max_grad_norm=1.0）、降低学习率、换 AdamW
梯度消失	Loss 降到一定程度后停滞	更好激活函数（ReLU/GELU）、残差连接、更强预训练模型